Curso: Before Open Ran
Aluna: Elizangela de Macedo Brito

Atividade Prática 1

Relatório de Execução

O objetivo da atividade: criação e execução de uma aplicação web em Python utilizando Docker e Docker Compose. O objetivo é demonstrar o uso de containers para empacotar e executar aplicações de maneira isolada, reproduzível e segura.

Conceito de containers: Containers são unidades de software que encapsulam uma aplicação e suas dependências em um ambiente isolado, permitindo execução consistente em qualquer ambiente.

Papel do Docker e Docker Compose: Docker é uma plataforma para criação e gerenciamento de containers. Docker Compose é uma ferramenta para definir e executar múltiplos containers com base em um arquivo de configuração YAML.

1. Estrutura do Projeto

```
mkdir projeto && cd projeto
touch Dockerfile docker-compose.yml run.py index.html
```

Output

```
user@ubuntu-lab:~/projeto$ ls -la
total 16
drwxr-xr-x  2 user user 4096 Sep 1 10:00 .
drwxr-xr-x  2 user user 4096 Sep 1 10:00 ..
-rw-r--r--  1 user user    0 Sep 1 10:00 Dockerfile
-rw-r--r--  1 user user    0 Sep 1 10:00 docker-compose.yml
-rw-r--r--  1 user user    0 Sep 1 10:00 index.html
-rw-r--r--  1 user user    0 Sep 1 10:00 run.py
```

2. Implementação dos Arquivos

Docherfile

```
FROM python:3.6
COPY run.py index.html /app/
USER www
VOLUME /log
EXPOSE 8000
```

docker-compose.yml

```yaml
version: '3'
services:
  web:
    build: .
    ports:
      - "8080:8000"
    volumes:
      - logs:/log
volumes:
  logs:
```

Output

```
user@ubuntu-lab:~/projeto$ cat docker-compose.yml
version: '3'
services:
  web:
    build: .
    ports:
      - "8080:8000"
    volumes:
      - logs:/log
 volumes:
   logs:
```

run.py

```python
#!/usr/bin/env python3
import http.server
import socketserver
import os

PORT = 8000
os.system('echo "Server started" > /log/http-server.log')

Handler = http.server.SimpleHTTPRequestHandler
with socketserver.TCPServer(("", PORT), Handler) as httpd:
    print(f"Serving at port {PORT}")
    httpd.serve_forever()
```

Output

```
user@ubuntu-lab:~/projeto$ cat run.py
#!/usr/bin/env python3
import http.server
import socketserver
import os

PORT = 8000
os.system('echo "Server started" > /log/http-server.log')

Handler = http.server.SimpleHTTPRequestHandler
with socketserver.TCPServer(("", PORT), Handler) as httpd:
    print(f"Serving at port {PORT}")
    httpd.serve_forever()
```

index.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>My App</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
</body>
</html>
```

Output

```
user@ubuntu-lab:~/projeto$ cat index.html
<!DOCTYPE html>
<html>
<head>
    <title>My App</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
</body>
 </html>
```

3. **Análise do Docker**

Dockerfile

```
FROM python:3.6
```

Utiliza Python 3.6

COPY run.py index.html /app/

```
COPY run.py index.html /app/
```

Copia os arquivos da aplicação para o diretório /app

USER www

```
USER www
```

Segurança: Executa o container como usuário www (não-root)

VOLUME /log

```
VOLUME /log
```

Persistência: Cria um volume para persistência de logs

EXPOSE 8000

```
EXPOSE 8000
```

Porta exposta: Expõe a porta 8000 para acesso externo

4. Build e Execução do Container

```
docker compose up --build
```

Output

```
user@ubuntu-lab:~/projeto$ docker compose up --build
[+] Building 0.5s (6/6) FINISHED
 => [internal] load build definition from Dockerfile                    0.0s
 => [internal] load .dockerignore                                      0.0s
 => [internal] load metadata for docker.io/library/python:3.6          0.4s
 => [1/3] FROM docker.io/library/python:3.6                            0.0s
 => [2/3] COPY run.py index.html /app/                                 0.1s
 => [3/3] RUN useradd -m www                                           0.1s
 => exporting to image                                                 0.0s
  => => exporting layers                                                0.0s
  => => writing image sha256:abc123def456                              0.0s
  => => naming to docker.io/library/projeto_web                         0.0s

 Creating projeto_web_1 ... done
 Attaching to projeto_web_1
 projeto_web_1  | Serving at port 8000
```

5. Validação da Aplicação

```
curl http://localhost:8080
```

Output

```
user@ubuntu-lab:~/projeto$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
    <title>My App</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
</body>
 </html>
```

6. Verificação do Container

```
docker compose ps
```

Output

```
user@ubuntu-lab:~/projeto$ docker compose ps
NAME              COMMAND                  SERVICE    CREATED          STATUS          PORTS
projeto_web_1     "/bin/sh -c '/usr/b…"    web        2 minutes ago    Up 2 minutes    0.0.0
```

7. Teste de imutabilidade da imagem

```
echo "<p>Alteração no host</p>" > index.html
```

Output

```
user@ubuntu-lab:~/projeto$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
    <title>My App</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
</body>
 </html>
```

Imagem imutável: A imagem Docker não é modificada após o build

Diferença build-time/runtime: Alterações no host não refletem na imagem

8. Rebuild da imagem

```
docker compose up --build
```

Output

```
user@ubuntu-lab:~/projeto$ docker compose up --build
[+] Building 0.5s (6/6) FINISHED
 => [internal] load build definition from Dockerfile                    0.0s
 => [internal] load .dockerignore                                       0.0s
 => [internal] load metadata for docker.io/library/python:3.6           0.4s
 => [1/3] FROM docker.io/library/python:3.6                             0.0s
 => [2/3] COPY run.py index.html /app/                                  0.1s
 => [3/3] RUN useradd -m www                                            0.1s
 => exporting to image                                                  0.0s
 => => exporting layers                                                 0.0s
 => => writing image sha256:abc123def456                               0.0s
 => => naming to docker.io/library/projeto_web                          0.0s

Creating projeto_web_1 ... done
Attaching to projeto_web_1
projeto_web_1  | Serving at port 8000
```

Recarregar navegador

Output

```
user@ubuntu-lab:~/projeto$ curl http://localhost:8080
<p>Alteração no host</p>
```

9. Validação de logs com volume

```
docker volume inspect projeto_logs
```

Output:

```
user@ubuntu-lab:~/projeto$ docker volume inspect projeto_logs
[
    {
        "CreatedAt": "2023-09-01T10:00:00.000000000Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/projeto_logs/_data",
        "Name": "projeto_logs",
        "Options": null,
         "Scope": "local"
     }
 ]
```

```
docker run --rm -v projeto_logs:/log python:3.6 cat /log/http-server.log
```

Output:

```
user@ubuntu-lab:~/projeto$ docker run --rm -v projeto_logs:/log python:3.6 cat /log/http-se
Server started
```

Persistência de dados: Logs são mantidos mesmo após reinicialização do container
Volume nomeado: Permite compartilhamento de dados entre containers
Independência do ciclo de vida: Volume permanece mesmo após container ser excluído.

10. Validação do usuário

```
docker compose exec web whoami
```

Output

```
user@ubuntu-lab:~/projeto$ docker compose exec web whoami
www
```

Segurança em containers: Execução como usuário não-root reduz riscos de segurança.
Execução como usuário não-root: Restringe acesso ao sistema de arquivos.

11. Finalização do ambiente

```
docker compose down
docker volume ls
```

Output

```
user@ubuntu-lab:~/projeto$ docker compose down
Stopping projeto_web_1 ... done
Removing projeto_web_1 ... done
Removing network projeto_default ... done

user@ubuntu-lab:~/projeto$ docker volume ls
DRIVER    VOLUME NAME
local     projeto_logs
```

```
docker compose down -v
docker volume ls
```

```
user@ubuntu-lab:~/projeto$ docker compose down -v
Removing volume projeto_logs

user@ubuntu-lab:~/projeto$ docker volume ls
DRIVER    VOLUME NAME
```

12. Modo de desenvolvimento

```
docker compose -f docker-compose.dev.yml up --build
```

Editar

```
echo "<p>Nova alteração</p>" > index.html
```

Recarregar navegador

Output

```
user@ubuntu-lab:~/projeto$ curl http://localhost:8080
<p>Nova alteração</p>
```

Atividade prática 1

Total de pontos 10/10

O e-mail do participante (**ebrito.lbv@gmail.com**) foi registrado durante o envio deste formulário.

✓ Considere um serviço web definido via Docker Compose a partir de um Dockerfile que copia os arquivos da aplicação para dentro da imagem (COPY run.py index.html /app/), utiliza um usuário não-root (www) e declara um volume nomeado para o diretório de logs (logs:/log). O servidor Python é iniciado com bind em 0.0.0.0:8000 e a porta 8080 do host é mapeada para a porta 8000 do container. *10/10

Um desenvolvedor edita o arquivo index.html no host e acessa http://localhost:8080 esperando ver as alterações. O conteúdo exibido no navegador não reflete as mudanças.

Qual é a causa correta desse comportamento?

- ○ O volume nomeado logs:/log impede que arquivos fora de /log sejam atualizados dentro do container.
- ○ O bind em 0.0.0.0 faz com que o servidor ignore arquivos modificados externamente durante a execução.
- ◉ Os arquivos da aplicação foram copiados para dentro da imagem em tempo de build. Alterações no host não afetam o container em execução — é necessário reconstruir a imagem com docker compose up --build. ✓
- ○ O Docker Compose armazena em cache o estado anterior do container e serve os arquivos a partir desse cache até que o container seja reiniciado com docker compose restart.